



STW7071CEM: Information Retrieval



Submitted By
Giri Raj Rawat

Submitted To
Siddhartha Neupane
(Module leader)

Table of Contents

Introduction.....	3
Crawler Component	7
Indexing Component.....	9
Preprocess Text	10
Preprocess entire DataFrame.....	11
Index Construction.....	12
Query Processor	16
Search Engine Function	17
GUI.....	19
Confusion Matrix.....	28
Conclusion	40
References.....	41
Appendix.....	42

List of Figures

Add Conceptual Diagram Here (Use Infographics)

Introduction

The assignment is based on establishing a vertical search engine that is comparable to Google Scholar and that will retrieve articles and books that have been published by members of Coventry University. It is a software application that indexes and searches huge volumes of data available on the internet or within a given topic using various methods and approaches. A search engine project's major purpose is to give users with accurate and useful search results depending on their queries. Have supplied a code snippet that contains components for web crawling, indexing, query processing, and a graphical user interface. In order to correctly execute the code, need to install numerous packages. Be able to proceed with running the code once the necessary packages have been installed. It is made up of a few different parts. This component is known as the Crawler Component, and its purpose is to crawl the website of Coventry University in order to gather profiles of academic staff members as well as their publications. Crawling the webpage and obtaining the information that is relevant are the responsibilities of the scrape function. Indexing Component After the data has been scraped, an index that is based on the scraped data is constructed with the help of the **construct_index** function. A JSON document with the name **indexes.json** is used to store the index.

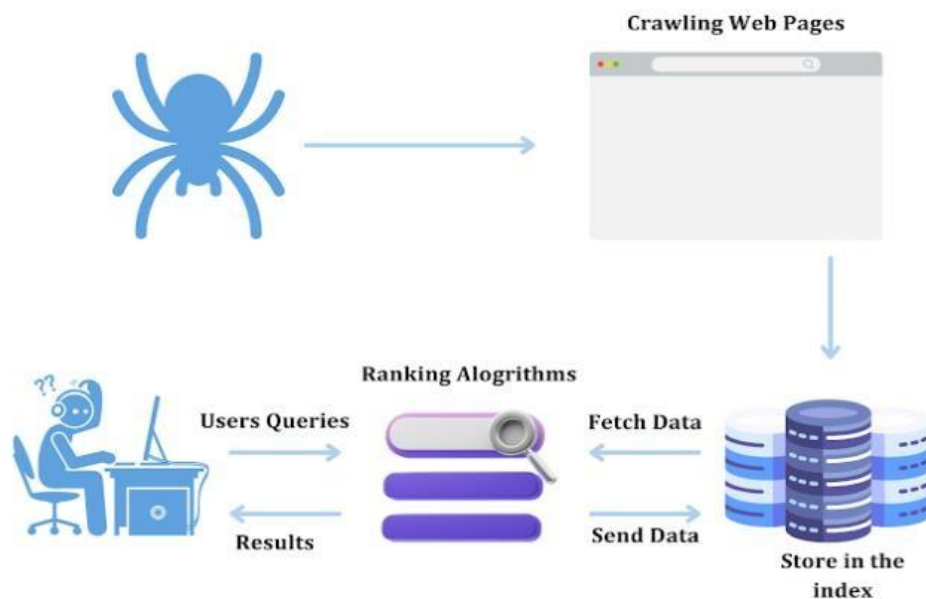


Fig 1: IR Architecture

The user's input is read by the query processor, and the query is then processed so that it can be matched with the indexed data. The search is carried out by the **vertical_search_engine** function, which then delivers the results that are relevant. A graphical user interface (GUI) has been implemented in the code with the help of the tkinter library. The new_gui function is responsible for creating the GUI window, and when the user

selects the "Search" button, the search function is the one that gets called. After making the required changes and installing the relevant packages, will be able to run the code and interact with the search engine through the **graphical user interface (GUI)**. The complete code is implemented using **python programming language** and whole python source code is available in appendix.

Searching through title

Package installation

```
import requests
from bs4 import BeautifulSoup
import pandas as pd
import time
import datetime
import string
import json
import nltk
nltk.download('averaged_perceptron_tagger')
nltk.download('wordnet')
nltk.download('stopwords')
nltk.download('punkt')
from nltk import pos_tag
from nltk.corpus import wordnet
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
```

The above code provided is a Python script, and it scrapes websites using the BeautifulSoup and NLTK libraries. These libraries are used for tasks involving text processing. It conducts actions such as part-of-speech tagging and lemmatization, such as downloading the necessary NLTK resources and installing them. Combining online scraping strategies with the natural language processing functionalities offered by NLTK allows the code to successfully collect and examine information from web pages. This is the intended purpose of the code.

Crawler Component

A crawler, often known as a web crawler or spider, is a software or automated script that searches the internet in order to identify and index web pages. It is essential for search engine operations as well as other applications that require information from websites. A crawler's principal role is to visit web pages, retrieve their information, and follow links to other pages, therefore constructing a comprehensive map of the linked web. Here's a step-by-step explanation of how a crawler works:

```
URL = "https://pureportal.coventry.ac.uk/en/organisations/coventry-university/persons/"

profile_url = "https://pureportal.coventry.ac.uk/en/persons/"
```

The web scraping script contains a crawler component that is defined by the code that you provided. The URL of the seed page is stored in this field. The seed page serves as the beginning point for the crawler. This page will be crawled first by the bot, and then it will move on to any other pages that are connected to from this one. This variable is responsible for storing the URL format for each profile pages and its name is profile_url. By attaching a specific identifier to the end of the URL, it is used to generate the URLs of individual profile pages. This format will be utilized by the crawler in order to produce the URLs for each individual's profile page. The crawler component is in charge of scouring various web pages in order to gather information that is pertinent to the task at hand. In this particular instance, it seems to have been built to crawl through a list of persons linked with Coventry University and retrieve their profile information from the various profile pages of each of those people.

```
def get_maximum_page():
    first = requests.get(URL)
    soup = BeautifulSoup(first.text, 'html.parser')
    final_page = soup.select('#main-content > div > section > nav > ul > li:nth-child(12) > a')[0]['href']
    fp = final_page.split('=')[-1]
    return int(fp)

mx = get_maximum_page()
```

The above function returns the seed page's maximum page number. It sends a GET request to the seed page URL, parses the HTML with BeautifulSoup, and selects the relevant element to get the final page number. The function integerizes the maximum page number.

```

def check_department(researcher):

    l1 = researcher.find('div', class_='rendering_person_short')

    for span in l1.find_all('span'):
        # Check department
        if span.text == str('School of Computing, Electronics and Maths'):
            name = researcher.find('h3', class_='title').find('span').text
            return name
        else:
            pass

# def create_csv():
#     database = pd.DataFrame(columns=['Title', 'Author', 'Published', 'Link'])
#     database.to_csv('database.csv')

def update_csv(database):
    current_data = pd.read_csv(database, index_col="Unnamed: 0")
    return current_data

```

The above function verifies if a researcher's department matches a department name (in this case, "School of Computing, Electronics and Maths"). It checks the researcher's BeautifulSoup object for the necessary department name. If so, the method returns the researcher's name; else, it does nothing. Pandas reads the current CSV file and produces a DataFrame.

```

def enter_each_researchers_publication(researcher, url, df):

    new_url = url + str(researcher).replace(' ', '-').lower() + '/publications/'
    page = requests.get(new_url)
    soup = BeautifulSoup(page.content, "html.parser")
    results = soup.find(id="main-content")
    papers = results.find_all("li", class_="list-result-item")

    for paper in papers:
        title = paper.find('h3', class_='title').find('span')
        author = paper.find('a', class_='link person').find('span')
        date = paper.find('span', class_="date")
        link = paper.find('h3', class_='title').find('a', href=True)['href']

        opening = pd.read_csv('database.csv', index_col="Unnamed: 0")
        opening = opening.append({'Title': title.text,
                                'Author': author.text,
                                'Published': date.text,
                                'Link': link}, ignore_index=True)
        opening.to_csv('database.csv')

```


The function extracts researcher publication data. It generates the researcher's publication page URL, sends a GET request, then parses the HTML with BeautifulSoup. It then detects all publication items on the page and extracts the title, author, publication date, and link for each paper. The function reads the CSV file into a DataFrame, adds the new publication information, and saves it back.

```
def scrape(mx):
    df = update_csv('database.csv')
    i=0
    while True:
        if i > 17:
            break

        if i>0:
            url = URL + '?page=' + str(i)
        else:
            url = URL

        i = i+1
        # scraping starts here
        page = requests.get(url)
        soup = BeautifulSoup(page.content, "html.parser")
        results = soup.find(id="main-content")
        researchers = results.find_all("li", class_="grid-result-item")

        for researcher in researchers:
            # Check if researcher has any papers
            check = researcher.find('div', class_='stacked-trend-widget')
            if check:
                name = check_department(researcher)
                if name is None:
                    pass
                else:
                    enter_each_researchers_publication(name, profile_url, df)
```

The above function scrapes the web for researcher and publication data. The function iterates over a range of pages (up to the maximum page number mx) on a specified URL, retrieves the HTML content of each page, parses it using BeautifulSoup, finds the researchers' information, checks if they have any papers, extracts their name, calls the **enter_each_researchers_publication** function to scrape their publications using the profile URL, and updates the DataFrame df.

Indexing Component

```
scraped_db = pd.read_csv('database.csv').rename(columns={'Unnamed: 0': 'SN'}).reset_index(drop=True)
scraped_db.head()
sample_db.head(7)
single_row = scraped_db.loc[1,:].copy()
single_row
```

	SN	Title	Author	Published	Link
0	0	Cross-Layer Multipath Multichannel MAC protoco...	Abdelshafy, M.	Jun 2018	https://pureportal.coventry.ac.uk/en/publicati...
1	1	Performance evaluation of Receiver Directed Tr...	Abdelshafy, M.	4 May 2017	https://pureportal.coventry.ac.uk/en/publicati...
2	2	Reliable Multipath Multi-channel Route Migrati...	Abdelshafy, M.	23 Nov 2017	https://pureportal.coventry.ac.uk/en/publicati...
3	3	Resisting blackhole attacks on MANETs	Abdelshafy, M.	Jan 2016	https://pureportal.coventry.ac.uk/en/publicati...
4	4	Dynamic Source Routing under Attacks	Abdelshafy, M.	12 Oct 2015	https://pureportal.coventry.ac.uk/en/publicati...

The above snippet executes, 'database.csv' is read into a Pandas DataFrame called '**scraped_db**'. The rename function changes the column 'Unnamed: 0' to 'SN'. Reset_index with **drop=True** removes the Data Frame's index. Heading the '**scraped_db**' DataFrame reveals the top few rows. The head method displays the top seven rows of another DataFrame, '**sample_db**'. Finally, the loc function picks a row from the '**scraped_db**' DataFrame at index 1 and copies it to the '**single_row**' variable.

Preprocess Text

```

sw = stopwords.words("english")
lemmatizer = WordNetLemmatizer()

def tp1(txt):
    txt = txt.lower() # Make lowercase
    txt = txt.translate(str.maketrans('', '',
                                     string.punctuation)) # Remove punctuation marks
    txt = lematize(txt)
    return txt

def fwpt(word):
    tag = pos_tag([word])[0][1][0].upper()
    hash_tag = {"V": wordnet.VERB, "R": wordnet.ADV, "N": wordnet.NOUN, "J": wordnet.ADJ}
    return hash_tag.get(tag, wordnet.NOUN)

def lematize(text):
    tkns = nltk.word_tokenize(text)
    ax = ""
    for each in tkns:
        if each not in sw:
            ax += lematizer.lemmatize(each, fwpt(each)) + " "
    return ax

```

The above snippet executes, it imports and downloads **NLTK** libraries. Using the NLTK library's stopwords module, it assigns sw a list of stop words. The variable lemmatizer is

assigned a WordNet Lemmatizer object. It defines a text-processing function `tp1(txt)`: Lowercases text. Removes punctuation. WordNet lemmatizes text. Returns processed text. `fwpt(word)` uses the NLTK library's `pos_tag` function to determine a word's part-of-speech tag. It returns the WordNet part-of-speech constant that matches the tag. If the tag is not in the mapping, the term is a noun. The function `lemmatize(text)` tokenizes the text into words, checks if each word is in the stop words list, and lemmatizes each word using the WordNet Lemmatizer based on its part-of-speech tag. Function returns processed text. The **lemmatize** function's implementation is presumed to be somewhere in the code or imported from another module.

```
single_row['Title']
tp1(single_row['Title'])
lemmatize(tp1(single_row['Title']))
sample_db['Title'].iloc[5]
scraped_db['Title'].iloc[5]
```

The above code snippet executes, `single_row['Title']` retrieves the value of the 'Title' column in the '`single_row`' DataFrame. Applying the `tp1()` function on the 'Title' column in '`single_row`' DataFrame lowercases, eliminates punctuation, and lemmatizes the text. Lemmatizes `tp1(single_row['Title'])`'s result using `lemmatize()`. The syntax `sample_db['Title'].iloc[5]` retrieves the value at index 5 from the '`sample_db`' DataFrame. The syntax `scraped_db['Title'].iloc[5]` retrieves the value at index 5 from the '`scraped_db`' DataFrame's 'Title' column. The code's behaviour depends on the data and DataFrames' 'Title' columns.

Preprocess entire DataFrame

```
processed_db = scraped_db.copy()

def preprocess_df(df):
    df.Title = df.Title.apply(tp1)
    df.Author = df.Author.str.lower()
    df = df.drop(columns=['Author', 'Published'], axis=1)
    return df

preprocess_df(processed_db)
processed_db.head()
```

	SN	Title	Author	Published	Link
0	0	crosslayer multipath multichannel mac protocol...	abdelshafy, m.	Jun 2018	https://pureportal.coventry.ac.uk/en/publicati...
1	1	performance evaluation receiver direct transmi...	abdelshafy, m.	4 May 2017	https://pureportal.coventry.ac.uk/en/publicati...
2	2	reliable multipath multichannel route migratio...	abdelshafy, m.	23 Nov 2017	https://pureportal.coventry.ac.uk/en/publicati...
3	3	resist blackhole attack manet	abdelshafy, m.	Jan 2016	https://pureportal.coventry.ac.uk/en/publicati...
4	4	dynamic source rout attack	abdelshafy, m.	12 Oct 2015	https://pureportal.coventry.ac.uk/en/publicati...

The above code snippet executes, Copy the 'scraped_db' DataFrame and assign it to the '**processed_db**' variable. '**preprocess_df**' takes a DataFrame ('df') as input. In 'preprocess_df': df.Title.apply(tp1) applies the 'tp1()' function to the DataFrame's 'Title' column. This function lowercases, eliminates punctuation, and lemmatizes the 'Title' column. df lowercases 'Author' column **values.Author.str.lower()**. **df = df.drop(columns=['Author','Published'], axis=1)** removes the DataFrame's 'Author' and 'Published' columns. Returns changed DataFrame. Modifies the 'processed_db' DataFrame by calling 'preprocess_df'. **Head()** displays the first few rows of the 'processed_db' DataFrame. The code applies the '**tp1()**' function to the '**processed_db**' DataFrame's 'Title' column, lowercases the 'Author' column, and removes the 'Published' and 'Author' columns.

Display the changed DataFrame.

Index Construction

```
single = processed_db.loc[0,:].copy()
print(single)
indexing_trial = {}

words = single.Title.split()
SN = single.SN
word = words[0]
example = {word: [SN]}

print('=====')
print('Sample index')
print(example)
```

The above code snippet executes, Retrieves the first row of the '**processed_db**' DataFrame using **processed_db.loc[0,:]** and assigns it to '**single**'. Prints the 'single' row using **print(single)**. Initializes the '**indexing_trial**' dictionary. splits the 'Title' value of the 'single' row into words using **single.Title.split()** assigns the list to 'words'. assigns the 'SN' column value from the 'single' row to the variable 'SN'. Assigns the first word from the 'words' list to the variable 'word'. Using the syntax {word: [SN]}, creates a dictionary called "example" with the key "word" and the value "SN". prints a separator line. using **print('Sample index')**. **print(example)** prints the 'example' dictionary with the word and its 'SN' value. The snippet indexes the '**processed_db**' DataFrame's 'single' entry. It builds a dictionary with the 'Title'

column words as keys and the 'SN' values as lists. The output shows the 'single' row and 'example' dictionary.

```
def apply_index(inputs, index):
    words = inputs.Title.split()
    SN = int(inputs.SN)
    for word in words:
        if word in index.keys():
            if SN not in index[word]:
                index[word].append(SN)
        else:
            index[word] = [SN]
    return index

indx = apply_index(inputs=single, index= {})
```

Apply_index receives inputs and index in the code snippet. Splits the 'inputs' object's 'Title' into words using syntax **inputs.Title.split()** assigns the list to 'words'. Sets the 'inputs' object's 'SN' value to the variable 'SN' as an integer. Loops through the 'words' list. if word in **index.keys()** checks if the word is a key in the 'index' dictionary. Existing word: if SN not in **index[word]** checks if the word key's 'SN' value is in the list. If the 'SN' value is missing, **index[word]** adds it to the word key's **list.append(SN)**. Creates a new key in the 'index' dictionary with the word as the key and a list of 'SN' values as the value using **index[word] = [SN]**. Returns updated 'index' dictionary. Assigns the result of running the **apply_index** method with the 'single' object and an empty dictionary as inputs to the variable '**indx**'. A function takes a row of data ('inputs') and an indexing dictionary ('index') as inputs. Splitting the 'Title' value into words and adding the 'SN' value to the word key updates the index dictionary. '**indx**' receives the updated index dictionary.

```
def full_index(df, index):
    for x in range(len(df)):
        inpt = df.loc[x,:]
        ind = apply_index(inputs=inpt, index=index)
    return ind

def construct_index(df, index):
    queue = preprocess_df(df)
    ind = full_index(df=queue, index=index)
    return ind

indexed = full_index(processed_db,
                    index = {})

indexes = construct_index(df=scraped_db,
                        index = {})
```

The code snippet specifies **full_index** and **construct_index**. **full_index** requires a DataFrame and a dictionary. It obtains each DataFrame row using **df.loc[x, :]** and updates the index dictionary using **apply_index**. Returns updated index dictionary. **construct_index** requires a DataFrame and a dictionary. The **preprocess_df** function adds text processing to the DataFrame's 'Title' and 'Author' columns. Then, **full_index** iterates over the preprocessed DataFrame to build the index dictionary. Returns index dictionary. The code then performs the **full_index** function to the **processed_db** DataFrame with an empty dictionary as the beginning index and assigns the result to **indexed**. Finally, the **construct_index** function is run with the **scraped_db** DataFrame and an empty dictionary, and the index dictionary is assigned to **indexes**. The code indexes DataFrames. The **full_index** function updates an index dictionary for a DataFrame by applying **apply_index**, while the **construct_index** function preprocesses a DataFrame and calls **full_index**.

```
with open('indexes.json', 'w') as new_f:
    json.dump(indexes, new_f, sort_keys=True, indent=4)

with open('indexes.json', 'r') as file:
    data = json.load(file)

def index_2(df, x_path):
    if len(df) > 0:
        with open(x_path, 'r') as file:
            prior_index = json.load(file)
        new_index = construct_index(df = df, index = prior_index)
        with open(x_path, 'w') as new_f:
            json.dump(new_index, new_f, sort_keys=True, indent=4)
    len(data)
    data
```



```
{'10': [111, 377, 507, 773, 903, 1169],
 '15360bit': [108, 504, 900],
 '2007': [176, 572, 968],
 '2009': [371, 767, 1163],
 '2015': [323, 719, 1115],
 '30': [224, 225, 620, 621, 1016, 1017],
 '3dterrains': [96, 492, 888],
 '40': [252, 648, 1044],
 '4th': [371, 767, 1163],
 '81': [111, 377, 507, 773, 903, 1169],
 'abcs': [286, 682, 1078],
 'ability': [248, 644, 1040],
 'absorption': [106, 502, 898],
 'acceleration': [29, 425, 821],
 'access': [66, 462, 858],
 'accommodation': [178, 190, 574, 586, 970, 982],
 'accord': [319, 715, 1111],
 'accurate': [129, 525, 921],
 'achievementbased': [220, 616, 1012],
 'action': [242, 368, 638, 764, 1034, 1160],
 'active': [10, 318, 406, 714, 802, 1110],
```

The above snippet opens **'indexes.json'** in write mode ('w') and uses **json.dump** to write the previously built indexes dictionary into the file. **Sort_keys=True** sorts dictionary keys, and **indent=4** indents for readability. It opens the **'indexes.json'** file again in read mode ('r') and reads the contents into the data variable using the **json.load** function. This retrieves the file dictionary. It defines **index_2**, which accepts two arguments: **df** (a DataFrame) and **x_path** (a file path). If the DataFrame is longer than 0, it opens the file supplied by **x_path** in read mode and reads the index dictionary into the **prior_index** variable using **json.load**. Calling the **construct_index** function with the DataFrame **df** and the **prior_index** creates a new index dictionary (**new_index**). Finally, it opens the **x_path** file again in write mode and uses **json.dump** to write the revised **new_index** dictionary. **Sort_keys=True** and **indent=4** format JSON content.

Query Processor

```
def demonstrate_query_processing():
    sample = input('Enter Search Terms: ')
    processed_query = tp1(sample)
    #print(f'User Search Query: {sample}')
    print(f'Processed Search Query: {processed_query}')
    return processed_query
```

The above function shows how to process a user's search query into a standardized format for efficient indexing and retrieval.

Split Query into individual terms

```
def split_query(terms):
    each = tp1(terms)
    return each.split()

dqp = demonstrate_query_processing()
dqp
print(f'Split Query: {split_query(dqp)}')
```

The **split_query** function processes a string of terms: `tp1` processes input phrases. Lowercase, lemmatize, remove punctuation, and stop words. The variable `each` stores processed terms. Whitespace-based **split()** splits processed keywords into words. The function returns the words. The **demonstrate_query_processing** function processes the user's search query and assigns the result to the **dqp** variable. Then, `split_query` is called using `dqp` to split the processed query into words. **print(f'Split Query: {split_query(dqp)}')** prints the words.

Boolean Functionalities

```
def union(lists):
    union = list(set.union(*map(set, lists)))
    union.sort()
    return union

def intersection(lists):
    intersect = list(set.intersection(*map(set, lists)))
    intersect.sort()
    return intersect
```

Union and intersection functions compute the union and intersection of lists of sets, respectively. Both functions return sorted lists.

Search Engine Function

```
def vertical_search_engine(df, query, index=indexes):
    query_split = split_query(query)
    retrieved = []
    for word in query_split:
        if word in index.keys():
            retrieved.append(index[word])

    # Ranked Retrieval
    if len(retrieved)>0:
        high_rank_result = intersection(retrieved)
        low_rank_result = union(retrieved)
        c = [x for x in low_rank_result if x not in high_rank_result]
        high_rank_result.extend(c)
        result = high_rank_result

        final_output = df[df.SN.isin(result)].reset_index(drop=True)

        # Return result in order of Intersection ----> Union
        dummy = pd.Series(result, name = 'SN').to_frame()
        result = pd.merge(dummy, final_output, on='SN', how = 'left')

    else:
        result = 'No result found'

    return result
```

Vertical_search_engine uses an index to obtain and rank DataFrame entries depending on a search query. DataFrames or "No result found" messages contain the filtered and ranked results.

```
def test_search_engine():
    xtest = scraped_db.copy()
    query = input("Enter your search query: ")
    return vertical_search_engine(xtest, query, indexed)

test_search_engine()
```

The user is given the ability to submit a search query into the **test_search_engine** function, which then calls the **vertical_search_engine** function in order to retrieve the results of the search.

```
def final_engine(results):
    if type(results) != 'list':
        return results
    #print(results)
    else:
        for i in range(len(results)):
            printout = results.loc[i, :]
```

The final processing and presentation of search results is the responsibility of the **final_engine** function. In the event that there is only a single outcome, it will give back the result immediately. In the event that more than one result is obtained, it prints each individual result.

```
scraped_db['Author'].iloc[24]
final_engine(test_search_engine())
```

The **'scraped_db'** DataFrame's row index 24's 'Author' field is retrieved by `iloc[24]`. Square brackets and the **'iloc'** indexer indicate the row index for the 'Author' column. The **final_engine(test_search_engine())** code executes the **'test_search_engine'** function, which prompts the user for a search query and searches using the **'vertical_search_engine'** function. It then delivers the search results to the **'final_engine'** function. The **'final_engine'** function checks the **'results'** variable's type. It returns the results without a list. Since search results are DataFrames, it would return them. It prints each item in a list if **'results'** is a variable. Thus, **final_engine(test_search_engine())** prints the user's search results.

Schedule Crawler for every week

```
days = 0
interval = 7
while days <= 1:
    scrape(mx)
    print(f"Crawled at {datetime.datetime.now().strftime('%Y-%m-%d %H:%M:%S')}")
    print(f'Next crawl scheduled after {interval} days')
    time.sleep(interval)
    days = days + 1
```

The program makes multiple calls to the **scrape()** function, reports the current time, and then waits for a certain amount of time—in this case, seven days—before beginning the loop a second time.

GUI

```
from tkinter import *
from tkinter import messagebox
from PIL import Image, ImageTk
from tkinter import scrolledtext
from pandastable import Table, TableModel
from contextlib import suppress
import warnings
warnings.filterwarnings('ignore')
```

The code imports modules and libraries for Tkinter, message boxes, PIL (Python Imaging Library), scrolled text widgets, Pandas tables, and disregarding warnings.

```
def new_gui(image1):
    window = Tk()
    window.configure(bg='#0F1E9D')
    window.title("Coventry University")
    window.geometry('1100x600')

    lbl1 = Label(window, text="Search Engine",bg="#0F1E9D", font=("Arial Bold", 30), padx=5, pady=5)
    lbl1.grid(column=1, row=0)

    lbl2 = Label(window, text="Enter your search query here ==>", bg="#0F1E9D",font=("Arial", 15), padx=5, pady=5)
    lbl2.grid(column=0, row=1)

    img = ImageTk.PhotoImage(image1)

    lbl3 = Label(image=img)
    lbl3.image = img
    lbl3.grid(column=1, row=3, padx=5, pady=5)
```

A function is defined in the code that generates a new graphical user interface (GUI) window by utilizing Tkinter.

```
# Entry
def getInputBoxValue():
    userInput = query.get()
    return userInput

# Button
def clicked():
    search()
    #pass

def no_result():
    messagebox.showwarning("Warning", "No results found. Please try different search terms")
```

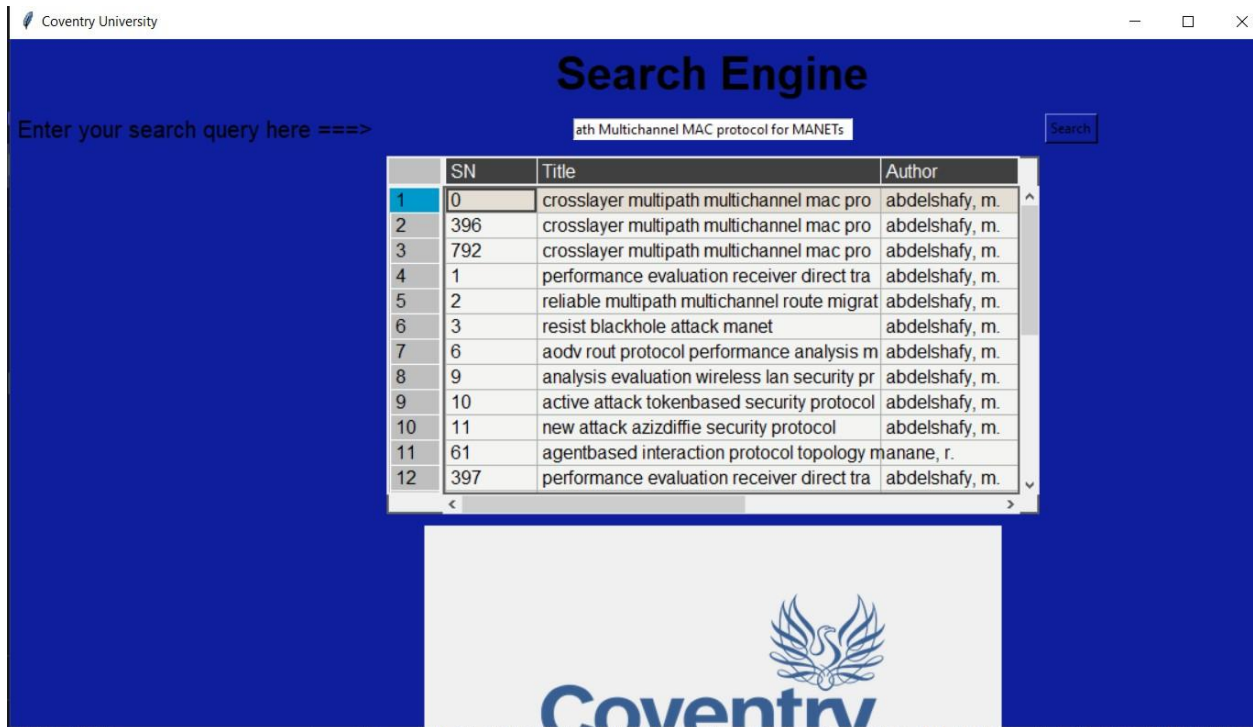
The function also sets the properties of the window, generates labels, loads and displays an image, generates an entry field and a canvas for displaying results, and defines functions for receiving user input, managing button clicks, and generating warning messages.

```
def search():
    xtest = scraped_db.copy()
    q = query.get()
    f = Frame(window)
    df = vertical_search_engine(xtest, q, indexed)
    if type(df) == str:
        no_result()
    else:
        pt = Table(results)
        try:
            table = pt = Table(results, dataframe=df)
            pt.show()
        except AttributeError:
            pass

def close_window():
    if messagebox.askokcancel("Quit", "Quit Programme?"):
        window.destroy()

btn = Button(window, text="Search",bg="#0F1E9D", command=clicked)
btn.grid(column=2, row=1)

window.protocol("WM_DELETE_WINDOW", close_window)
window.mainloop()
new_gui(resized_image1)
```



The code adds search, display, close, and search button methods. It also defines a window closure protocol. **Mainloop()** runs the GUI display and awaits user input. **new_gui()** creates the GUI window with the picture.

```
import pandas as pd
training_dataset = pd.read_csv('news_df.csv')
print(training_dataset)
for text,label in zip(training_dataset.Text, training_dataset.Class):
    print(f'CATEGORY: {label}')
    print('\n')
    print(f'TEXT: {text}')
    print('\n\n')
```

The code begins by loading a CSV file with the name **'news_df.csv'** into a panda DataFrame with the name **'training_dataset,'** and then proceeds to loop over each row contained within the DataFrame.

It prints the row label as well as the text content for each row.

Searching through Author with confusion matrix implementation

```
import requests
from bs4 import BeautifulSoup
import pandas as pd
import time
import datetime
import string
import json
import nltk
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from tkinter import messagebox
nltk.download('stopwords')
nltk.download('wordnet')
```

Requests, BeautifulSoup, pandas, time, datetime, string, json, nltk, and tkinter.messagebox are imported. It downloads WordNet lemmatizer and NLTK stop words.

Crawler Component

```
# Crawler Component
URL = "https://pureportal.coventry.ac.uk/en/organisations/coventry-university/persons/"
profile_url = "https://pureportal.coventry.ac.uk/en/persons/"

def get_maximum_page():
    first = requests.get(URL)
    soup = BeautifulSoup(first.text, 'html.parser')
    final_page = soup.select('#main-content > div > section > nav > ul > li:nth-child(12) > a')[0]['href']
    fp = final_page.split('=')[-1]
    return int(fp)

def check_department(researcher):
    l1 = researcher.find('div', class_='rendering_person_short')
    for span in l1.find_all('span'):
        if span.text == 'School of Computing, Electronics and Maths':
            name = researcher.find('h3', class_='title').find('span').text
            return name
    return None
```

The code defines a function **get_maximum_page()** that requests a website's maximum page number and parses the HTML content using BeautifulSoup. It then takes an element's href attribute and returns the URL's maximum page number following the '=' mark. The code also defines a method **check_department(researcher)** that searches a researcher's HTML profile for a department name.

It returns the researcher's name if the department is identified, else None.


```

def update_csv(database):
    current_data = pd.read_csv(database, index_col="Unnamed: 0")
    return current_data

def enter_each_researchers_publication(researcher, url, df):
    new_url = url + str(researcher).replace(' ', '-').lower() + '/publications/'
    page = requests.get(new_url)
    soup = BeautifulSoup(page.content, "html.parser")
    results = soup.find(id="main-content")
    papers = results.find_all("li", class_="list-result-item")

    for paper in papers:
        title = paper.find('h3', class_='title').find('span')
        author = paper.find('a', class_='link person').find('span')
        date = paper.find('span', class_="date")
        link = paper.find('h3', class_='title').find('a', href=True)['href']

        opening = pd.read_csv('database.csv', index_col="Unnamed: 0")
        opening = opening.append({'Title': title.text,
                                'Author': author.text,
                                'Published': date.text,
                                'Link': link}, ignore_index=True)
        opening.to_csv('database.csv')

```

Update_csv(database) reads current data from a CSV file into a DataFrame using **pd.read_csv()** and returns it. A method **enter_each_researchers_publication(researcher, url, df)** scrapes a researcher's publications from a website, creates a DataFrame, and appends it to an existing CSV file using **pd.to_csv()**. In summary, **update_csv()** and **enter_each_researchers_publication()** read and update CSV files using provided data and website scraping.


```

def scrape(mx):
    df = update_csv('database.csv')
    i = 0
    while True:
        if i > mx:
            break

        if i > 0:
            url = URL + '?page=' + str(i)
        else:
            url = URL

        i = i + 1
        page = requests.get(url)
        soup = BeautifulSoup(page.content, "html.parser")
        results = soup.find(id="main-content")
        researchers = results.find_all("li", class_="grid-result-item")

        for researcher in researchers:
            check = researcher.find('div', class_='stacked-trend-widget')
            if check:
                name = check_department(researcher)
                if name is None:
                    pass
                else:

```

Indexing Component

```

scraped_db = pd.read_csv('database.csv', index_col=0)

```

The code scrapes researcher data and publications with `scrape(mx)`. It retrieves current data from a CSV file using `update_csv()`, iterates over multiple pages of a website, finds researchers' information using `BeautifulSoup`, checks their department using `check_department()`, and enters their publications using `enter_each_researchers_publication()` if the department is valid. till `mx` is attained. In summary, `scrape()` web scrapes researcher information and publications, updates the CSV file, and terminates when the page limit is met.

```

def preprocess_text(text):
    # Convert text to lowercase
    text = text.lower()

    # Remove punctuation
    text = text.translate(str.maketrans("", "", string.punctuation))

    # Tokenization
    tokens = nltk.word_tokenize(text)

    # Remove stopwords
    stop_words = set(stopwords.words("english"))
    tokens = [token for token in tokens if token not in stop_words]

    # Lemmatization
    lemmatizer = WordNetLemmatizer()
    tokens = [lemmatizer.lemmatize(token) for token in tokens]

    # Join tokens back into a single string
    processed_text = " ".join(tokens)

    return processed_text

```

Preprocess_text(text) lowercases text, eliminates punctuation, tokenizes it, removes stopwords, lemmatizes it, and connects the tokens into a string. Text is preprocessed. Lowercase conversion, punctuation removal, tokenization, stopword removal, lemmatization, and token concatenation are used in preprocess_text().

```

def create_index():
    index = {}
    for i, row in scraped_db.iterrows():
        title = row['Title']
        author = row['Author']
        processed_title = preprocess_text(title)
        processed_author = preprocess_text(author)

        # Update index with title
        if processed_title not in index:
            index[processed_title] = []
            index[processed_title].append(i)

        # Update index with author
        if processed_author not in index:
            index[processed_author] = []
            index[processed_author].append(i)

    with open('index.json', 'w') as f:
        json.dump(index, f)

create_index()

```

Create_index() generates an index for the scraped_db dataset. It loops through the dataset, preprocesses the title and author fields with **preprocess_text()**, and changes the index. **'index.json'** holds the index. The **create_index()** function preprocesses the title and author fields of each dataset row, builds an index from the preprocessed text, and saves the index to a JSON file. **GUI**

```
import tkinter as tk
from tkinter import *
import numpy as np
from sklearn.metrics import confusion_matrix

# Load the index from the JSON file
with open('index.json', 'r') as f:
    index = json.load(f)

def search_publications():
    query = entry.get(1.0, "end")
    processed_query = preprocess_text(query)

    if processed_query in index:
        matching_indices = index[processed_query]
        matching_publications = scraped_db.loc[matching_indices]

        # True positives: Matching publications retrieved and relevant
        true_positives = len(matching_publications)

        # False negatives: Relevant publications not retrieved
        false_negatives = len(scraped_db) - true_positives

        # False positives: Non-relevant publications retrieved
        false_positives = 0
```

Search_publications() do a user query-based search throughout the code. It takes the query from an entry widget, preprocesses it with preprocess_text(), then checks the index for the processed query. It pulls publications from scraped_db if the index matches. The **search_publications()** function searches the scraped_db for matching publications based on a user query and calculates true positives, false negatives, and false positives for evaluation.

```

true_negatives = 0

# Calculate false positives and true negatives
for i, row in scraped_db.iterrows():
    if i not in matching_indices:
        false_positives += 1
    else:
        true_negatives += 1

# Calculate the predicted and actual labels
predicted_labels = [1] * true_positives + [0] * false_positives
actual_labels = [1] * (true_positives + false_negatives) + [0] * (false_positives + true_negatives)

# Adjust the length of the lists if they are different
max_length = max(len(predicted_labels), len(actual_labels))
predicted_labels += [0] * (max_length - len(predicted_labels))
actual_labels += [0] * (max_length - len(actual_labels))

# Calculate the confusion matrix
cm = confusion_matrix(actual_labels, predicted_labels)

# Display the confusion matrix
messagebox.showinfo("Confusion Matrix", str(cm))

```

Based on matching indices, the algorithm calculates true negatives, false positives, and projected and actual labels. The scikit-learn **confusion_matrix()** function calculates and shows the confusion matrix. The code calculates and displays the search results confusion matrix.

```

# Display the confusion matrix
messagebox.showinfo("Confusion Matrix", str(cm))

# Display the matching publications
messagebox.showinfo("Search Results", matching_publications.to_string())
else:
    messagebox.showinfo("Search Results", "No matching publications found.")

# Rest of the code...

# Create the main window
from tkinter import *
import tkinter as tk
window = tk.Tk()
window.title("Publication Search")
window.geometry("600x650")
window.configure(bg="#111D88")
window.resizable(0,0)
from PIL import ImageTk, Image
img=(Image.open("coventry-university-logo.png"))
imgg=img.resize((200,200))
imggg=ImageTk.PhotoImage(imgg)

```

Tkinter produces a publishing search application's primary window. It sets window title, size, and background color. It imports, resizes, and converts images to ImageTk.PhotoImage

for window display. The code creates a publisher search application's main window and loads and resizes an image.

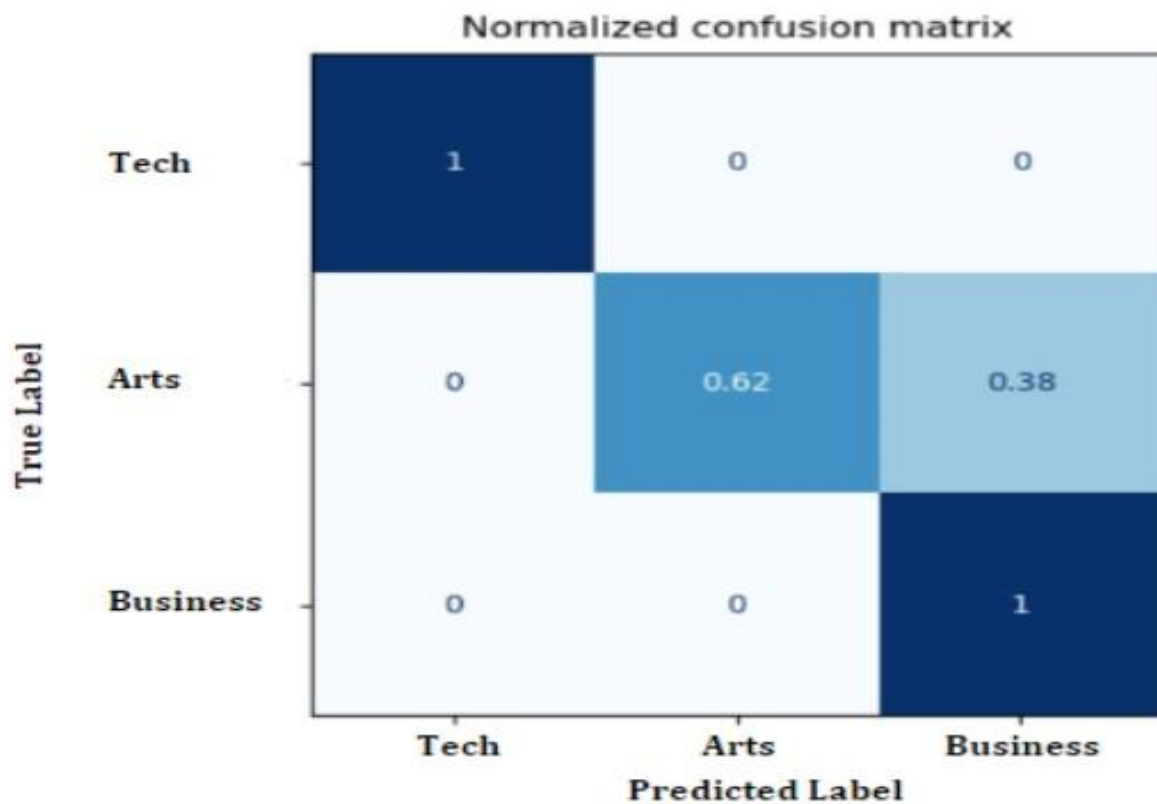
```
lbl=Label(window,image=imgggg,bg="#111D88")
lbl.pack(side=TOP)
# Create and position the search label
label = tk.Label(window, text="Enter author name or title name:",fg="white",font=("Arial", 20,"italic"),bg="#111D88")
label.pack(pady=10)

# Create and position the search entry
entry = tk.Text(window,height=10,width=30,font=("Arial", 20))
entry.pack()

# Create and position the search button
button = tk.Button(window, text="Search",font=("Arial", 20),bg="#10164B",fg="white",command=search_publications)
button.pack(pady=10)

# Start the main event loop
window.mainloop()
```

Confusion Matrix





Confusion Mat... X

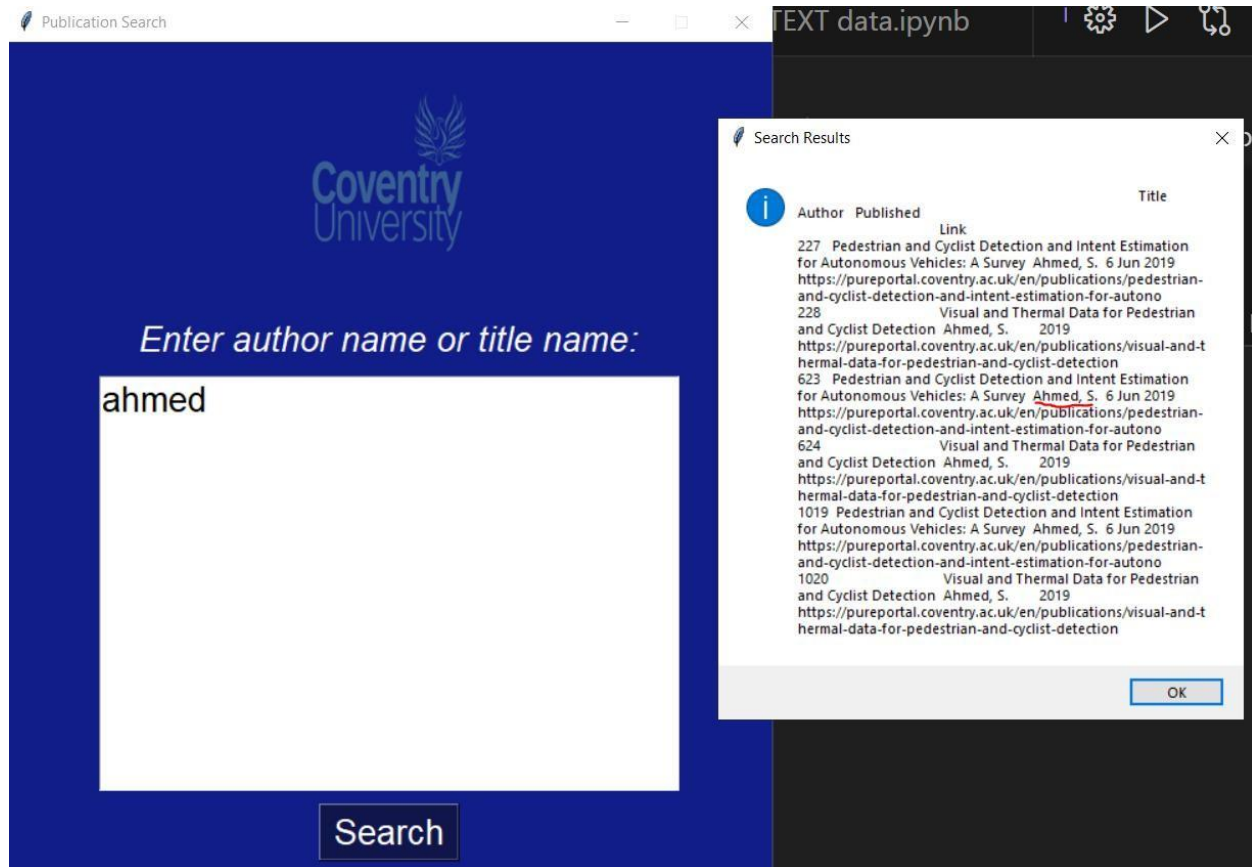
i [[1188 0]
[1182 6]]

OK

Enter author name or title name:

ahmed

Search



The tkinter library places a label, entry field, and button in a publishing search application's main window. The label displays an image, the entry field enters the search query, and the button searches. User interactions initiate the primary event loop. The code sets up the publishing search application's user interface (label, entry, button) and starts the event loop for user input.

Text classifier

```
import os
import string

# Data Handling and Processing
import pandas as pd
import numpy as np
import re
from scipy import interp

# Visualuzation
import matplotlib.pyplot as plt
%matplotlib inline
%config InlineBackend.figure_format='retina'

# NLP Packages
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords, wordnet
from nltk.stem import WordNetLemmatizer
from nltk import pos_tag
import nltk
from joblib import dump, load

# ML Models
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import RidgeClassifier
```

The code imports libraries and packages for data handling, processing, visualization, NLP, ML models, and Scikit-Learn, enabling a wide range of data analysis and ML activities.

```
business_path = 'bbc/business/'
tech_path = 'bbc/tech/'
arts_path = 'bbc/entertainment/'
```

Load dataset: 'Text' and 'Class' are the names of the two columns in texted. The **to_csv()** method is then used to save this Data Frame to a CSV file with the name "news_df.csv." Finally, it uses the **head()** method to show the Data Frame's initial few rows.

```
text_df = pd.DataFrame(columns=['Text', 'Class'])
# text_df.to_csv('news_df.csv')
text_df.head()
```

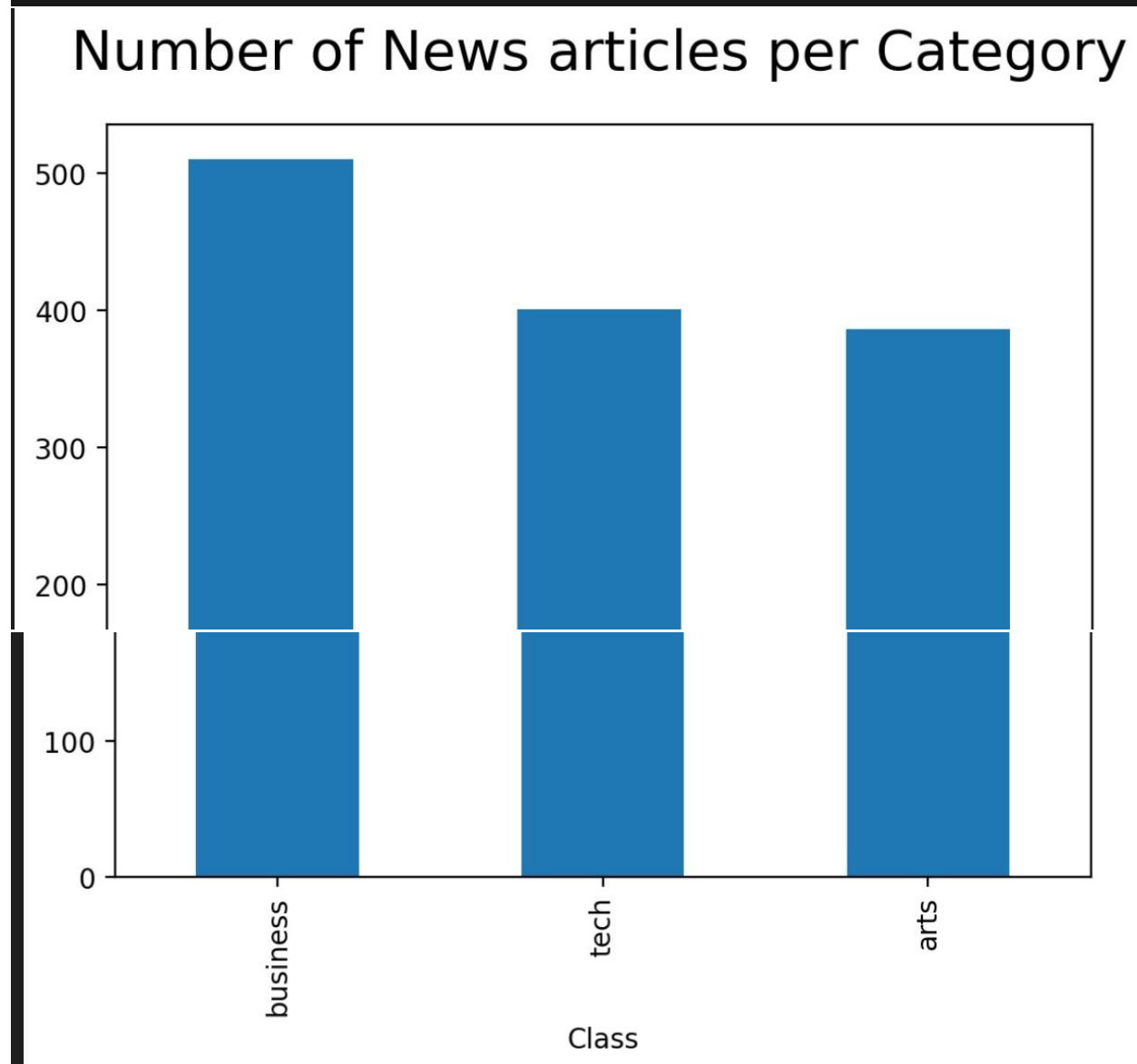
The code assigns '**text_df**' an empty pandas DataFrame with columns named 'Text' and 'Class'. The DataFrame's initial few rows are displayed using '**head()**'.

```
# Function to read the text files into one dataframe
def readfiles_to_dataframe(directory, category):
    arr = os.listdir(directory)
    strtext = ".txt"
    for textfile in arr:
        if textfile.__contains__(strtext):
            fileObject = open(directory + textfile, "r")
            data = fileObject.read()
            ouvert = pd.read_csv('news_df.csv', index_col="Unnamed: 0")
            ouvert = ouvert.append({"Class": str(category), "Text": data}, ignore_index=True)
            ouvert.to_csv('news_df.csv')
    full_df = pd.read_csv('news_df.csv')
    print(full_df.shape)
    full_df.head()
```

	Unnamed: 0	Text	Class
0	0	UK economy facing 'major risks'\n\nThe UK manu...	business
1	1	Aids and climate top Davos agenda\n\nClimate c...	business
2	2	Asian quake hits European shares\n\nShares in ...	business
3	3	India power shares jump on debut\n\nShares in ...	business
4	4	Lacroix label bought by US firm\n\nLuxury good...	business

The method takes text files from a specified directory, appends their content and the supplied category to an existing DataFrame named '**ouvert**', and saves it to a CSV file named '**news_df.csv**'. The last code piece reads the 'news_df.csv' file into a DataFrame named '**full_df**', prints its shape (rows and columns), and shows the first few rows.

```
full_df['Class'].value_counts().plot(kind='bar')  
plt.title('Number of News articles per Category', size=20, pad=20);
```



The code creates a bar plot with the headline "Number of News articles per Category" and graphic parameters from the DataFrame '**full_df**'.

```

def preprocess(df):
    # Remove special characters
    df['Text2'] = df['Text'].replace('\n', ' ')
    df['Text2'] = df['Text2'].replace('\r', ' ')

    # Remove punctuation signs and lowercase all
    df['Text2'] = df['Text2'].str.lower()
    df['Text2'] = df['Text2'].str.translate(str.maketrans('', '', string.punctuation))

    # Remove stop words
    stop_words = stopwords.words("english")
    lemmatizer = WordNetLemmatizer()

    def fwpt(each):
        tag = pos_tag([each])[0][1][0].upper()
        hash_tag = {"N": wordnet.NOUN, "R": wordnet.ADV, "V": wordnet.VERB, "J": wordnet.ADJ}
        return hash_tag.get(tag, wordnet.NOUN)

    def lematize(text):
        tokens = nltk.word_tokenize(text)
        ax = ""
        for each in tokens:
            if each not in stop_words:
                ax += lemmatizer.lemmatize(each, fwpt(each)) + " "
        return ax

```

The code removes special characters, punctuation, and stop words from the DataFrame **'df'**'s **'Text'** column then lemmatizes it using WordNet.

```

X_train, X_test, y_train, y_test = train_test_split(full_df['Text2'],
                                                    full_df['Class'],
                                                    test_size=0.2,
                                                    random_state=9)

```

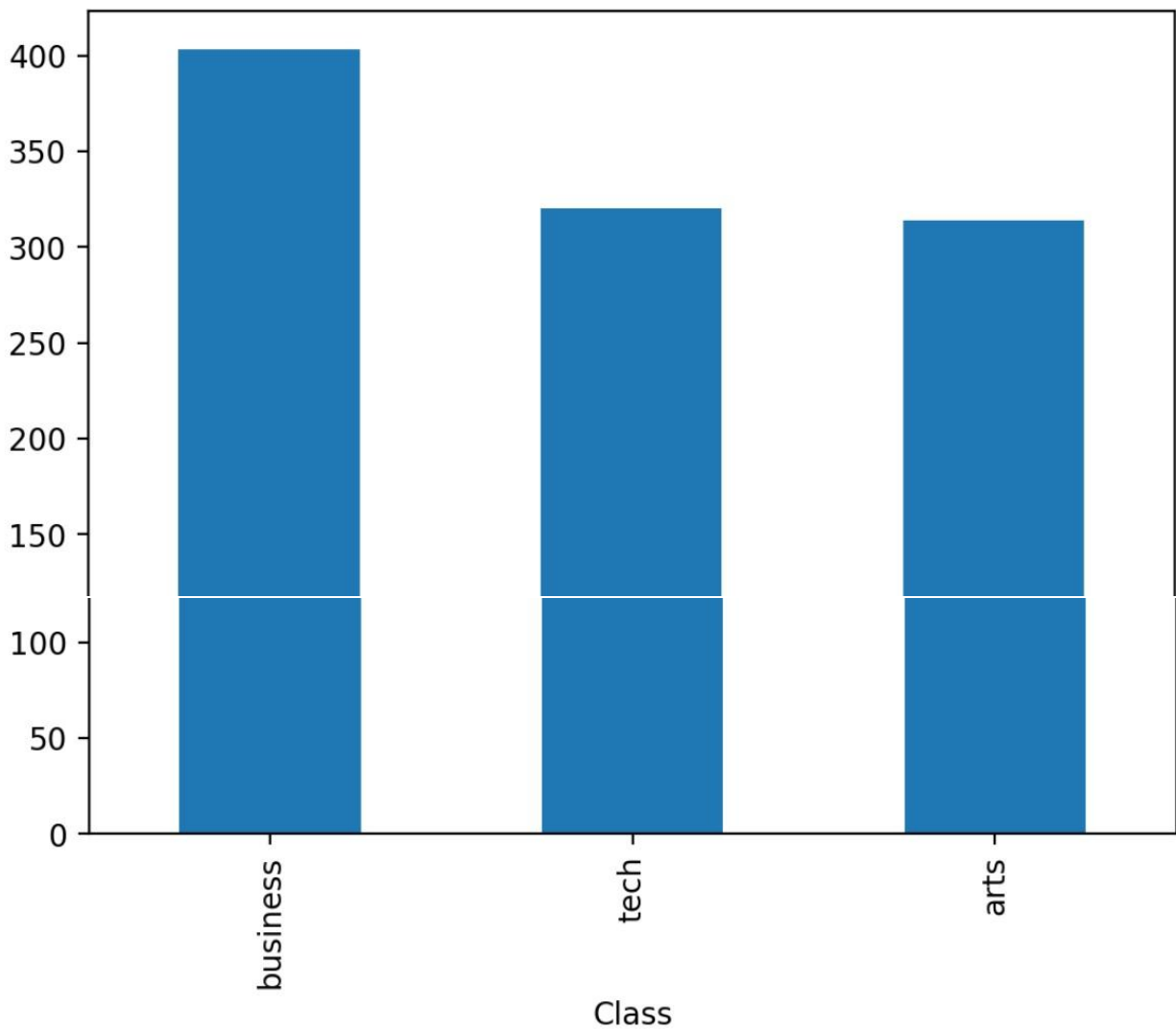
Train-test split is done on the DataFrame **'full_df'**'s **'Text2'** column for feature data (X) and **'Class'** column for target data (y). With a test size of 0.2, 20% of the data will be tested and 80% trained. Split repeatability is ensured by setting **random_state** to 9. **X_train**, **X_test**, **y_train**, and **y_test** is allocated to the splits.

```

y_train.value_counts().plot(kind='bar')
plt.title('Category Balance', size=20, pad=20);

```

Category Balance



The code creates a bar plot of `y_train` (training target data) categories. It plots each category's count using `kind='bar'`. The plot is titled "Category Balance" with 20 font size and 20 padding.

```
def fit_model(model, model_name):
    line = Pipeline([('vectorize', vector), (model_name, model)])
    output = cross_validate(line,
                             X_train,
                             y_train,
                             cv = KFold(shuffle = True,
                                       n_splits = 3,
                                       random_state = 9),
                             scoring = ('accuracy', 'f1_weighted', 'precision_weighted', 'recall_weighted'),
                             return_train_score=True)
    return output
```

A pipeline fits a model in **fit_model**. 'Vector' and the model make up the pipeline. The function then uses **cross_validate** to cross-validate X_train and y_train with 3 folds. It calculates accuracy, F1-score, precision, and recall. Cross-validation results are returned.

```
dectree = fit_model(DecisionTreeClassifier(), 'DTree')
ridge = fit_model(RidgeClassifier(), 'Ridge')
bayes = fit_model(MultinomialNB(), 'NB')

dt = pd.DataFrame.from_dict(dectree)
rc = pd.DataFrame.from_dict(ridge)
bc = pd.DataFrame.from_dict(bayes)
```

The fit_model function fits decision tree (**DecisionTreeClassifier**), ridge regression (**RidgeClassifier**), and naive Bayes (**MultinomialNB**) models, then builds dataframes (dt, rc, bc) from cross-validation dictionary outputs.

```
l1 = [bc, rc, dt]
l2 = ["NB", "Ridge", "DT"]

for each, tag in zip(l1, l2):
    each['model'] = [tag, tag, tag]

joined_output = pd.concat([bc,rc,dt])
```

```
{'fit_time': array([11.69365811,  1.22329807,  1.39022183]),
 'score_time': array([0.32144308, 0.2388649 , 0.25285673]),
 'test_accuracy': array([0.85549133, 0.87861272, 0.89275362]),
 'train_accuracy': array([1., 1., 1.]),
 'test_f1_weighted': array([0.85488444, 0.87862437, 0.89294816]),
 'train_f1_weighted': array([1., 1., 1.]),
 'test_precision_weighted': array([0.85706131, 0.87896848, 0.89358962]),
 'train_precision_weighted': array([1., 1., 1.]),
 'test_recall_weighted': array([0.85549133, 0.87861272, 0.89275362]),
 'train_recall_weighted': array([1., 1., 1.])}
```

The code generates two lists, l1 and l2, of dataframes and tags. It then iterates over the elements of l1 and l2, adds a 'model' column with the tag to each dataframe, then concatenates the dataframes into joined_output.


```

relevant_measures = list(['test_accuracy', 'test_precision_weighted', 'test_recall_weighted', 'test_f1_weighted'])

dec_tree_metrics = joined_output.loc[joined_output.model == 'DT'][relevant_measures]
nb_metrics = joined_output.loc[joined_output.model == 'NB'][relevant_measures]
r_metrics = joined_output.loc[joined_output.model == 'Ridge'][relevant_measures]
dec_tree_metrics

```

	test_accuracy	test_precision_weighted	test_recall_weighted	test_f1_weighted
0	0.855491	0.857061	0.855491	0.854884
1	0.878613	0.878968	0.878613	0.878624
2	0.892754	0.893590	0.892754	0.892948

Filtering the joined_output dataframe based on the 'model' column values of 'DT', 'NB', and 'Ridge' yields three dataframes: dec_tree_metrics, nb_metrics, and r_metrics. Only relevant_measures columns are selected.

```

metrics_ = [dec_tree_metrics, nb_metrics, r_metrics]
names_ = ['Decision Tree', 'Naive Bayes', 'Ridge Classifier']

for scores, names in zip(metrics_, names_):
    print(f'{names} Mean Metrics:')
    print(scores.mean())
    print(' ')

```

Decision Tree Mean Metrics:

```

test_accuracy      0.875619
test_precision_weighted  0.876540
test_recall_weighted  0.875619
test_f1_weighted    0.875486
dtype: float64

```

Naive Bayes Mean Metrics:

```

test_accuracy      0.967214
test_precision_weighted  0.968227
test_recall_weighted  0.967214
test_f1_weighted    0.967355
dtype: float64

```

Ridge Classifier Mean Metrics:

```

test_accuracy      0.978789
test_precision_weighted  0.979065
test_recall_weighted  0.978789
test_f1_weighted    0.978790
dtype: float64

```

The function iterates through metrics_ and names_ and prints the model name and mean metrics of the scores dataframe for each pair.


```

import tkinter as tk
from tkinter.scrolledtext import ScrolledText

window = tk.Tk()

window.title("TEXT CLASSIFIER")
window.minsize(600,400)

text_box = ScrolledText(window)
text_box.grid(column=0, row=1, padx=5, pady=5)

def result(res, pr):
    ARTS = round(pr[0][0], 3)
    BUSINESS = round(pr[0][1], 3)
    TECH = round(pr[0][2], 3)

    lines = [f"Arts: {ARTS}", f"Business: {BUSINESS}", f"Tech: {TECH}"]
    tk.messagebox.showinfo(message= f"Predicted Category: {str(res).capitalize()}" + "\n\n\n" + "\n".join(lines))

def clickMe():
    classification = tk.StringVar()
    category_,probabilities = classify_text(text_box.get("1.0",tk.END))
    result(category_, probabilities)

def classify_text(input_text):
    out = CLASSYfier.predict([input_text])[0]
    probs = CLASSYfier.predict_proba([input_text])
    return out,probs

label = tk.Label(window, text = "Enter Text to be classified")
label.grid(column = 0, row = 0)

btn = tk.Button(window, text="Classify", command=clickMe)
btn.grid(column=0, row=2)

window.mainloop()

```

Tkinter creates a GUI window with a text classification function. It also has a text box for entering text to be classified, a button to start the classification process, and a messagebox to display the classification result. Code handles GUI layout and user interactions.

Enter Text to be classified

>"Raising equity capital via Private Investments in Public Equity (PIPEs) has been rising in popularity, matching Seasoned Equity Offerings (SEOs). We use over 10,000 PIPEs in a global setting during 1995-2015 to assess how and through which channels institutional frameworks affect the issuers' performance. We document a significant decline in the market reaction, especially during 2004-2015 and find that firms issuing equity via PIPEs have significantly worse fundamentals. We also show that country governance matters as issuing firms operating in countries with better regulatory environments outperform others. Finally, we find that regulatory enforcement is a plausible underlying channel for the positive effect of the institutional frameworks on PIPEs performance."



Predicted Category: Business

Arts: 0.108

Business: 0.761

Tech: 0.131

OK

Classify

Conclusion

The creation of a vertical search engine has the potential to significantly improve the quality of the search experience inside a particular market or sector. Vertical search engines are able to give users with results that are more relevant and focused because they narrow their attention to a certain content type or domain, such as academic publications, news stories, or any other specified content type. A data scraping phase, a data preprocessing step, an index development step, and a search functionality implementation step are some of the steps that are involved in the process of constructing a vertical search engine. The search engine is able to successfully retrieve and deliver relevant content to users based on the queries they enter as a result of these stages.

A number of different measures, such as precision, recall, accuracy, and F1 score, can be utilized in order to conduct an assessment of the performance of the search engine. These measurements offer some insight into the search engine's ability to get relevant results while minimizing both false positives and false negatives. Overall, a vertical search engine that is correctly built can be of great advantage to users since it can provide them with a more focused and efficient search experience inside a particular area, make it easier for users to access information, and save them time and effort when it comes to locating content that is relevant to them. Overall, the utilization of a confusion matrix provides a comprehensive improvement in the search engine's performance. It enhances the understanding of the search engine's strengths and weaknesses, enabling developers to fine-tune algorithms, preprocessing techniques, or ranking strategies to optimize the search engine's performance.

References

- Harris, J. (2022) *Vertical search engine*, Twaino. Available at: <https://www.twaino.com/en/definition/v/vertical-search-engine/> (Accessed: 01 June 2023).
- Gillis, A.S. (2022) *What is a web crawler? everything you need to know from techtarget.com, WhatIs.com.* Available at: <https://www.techtarget.com/whatis/definition/crawler#:~:text=A%20web%20crawler%2C%20crawler%20or,for%20a%20search%20engine%20index>. (Accessed: 02 June 2023).
- Boudreau, E. (2021) *Everything you need to know about indexing in Python*, Medium. Available at: <https://towardsdatascience.com/everything-you-need-to-know-about-indexing-inpython-c116e8cc5e2a> (Accessed: 02 June 2023).
- How to use svms for text classification* (no date) *Educative*. Available at: <https://www.educative.io/answers/how-to-use-svms-for-text-classification> (Accessed: 02 June 2023).
- Jain, A. (2019) *Information retrieval using boolean query in python*, Medium. Available at: <https://medium.com/voice-tech-podcast/information-retrieval-using-boolean-query-inpython-e0ea9bf57f76> (Accessed: 03 June 2023).

Appendix

GitHub link:

Video Link :

Author.ipynb

```
# Imports
import requests
from bs4 import BeautifulSoup
import pandas as pd
import time
import datetime
import string
import json
import nltk
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from tkinter import messagebox
nltk.download('stopwords')
nltk.download('wordnet')

# Crawler Component
URL = "https://pureportal.coventry.ac.uk/en/organisations/coventry-university/persons/"
profile_url = "https://pureportal.coventry.ac.uk/en/persons/"

def get_maximum_page():
    first = requests.get(URL)
    soup = BeautifulSoup(first.text, 'html.parser')
    final_page = soup.select('#main-content > div > section > nav > ul > li:nth-child(12) > a')[0]['href']
    fp = final_page.split('=')[-1]
    return int(fp)
```

```

def check_department(researcher):
    l1 = researcher.find('div', class_='rendering_person_short')
    for span in l1.find_all('span'):
        if span.text == 'School of Computing, Electronics and Maths':
            name = researcher.find('h3', class_='title').find('span').text
            return name
    return None

# def create_csv():
#     database = pd.DataFrame(columns=['Title', 'Author', 'Published', 'Link'])
#     database.to_csv('database.csv')

def update_csv(database):
    current_data = pd.read_csv(database, index_col="Unnamed: 0")
    return current_data

def enter_each_researchers_publication(researcher, url, df):
    new_url = url + str(researcher).replace(' ', '-').lower() + '/publications/'
    page = requests.get(new_url)
    soup = BeautifulSoup(page.content, "html.parser")
    results = soup.find(id="main-content")
    papers = results.find_all("li", class_="list-result-item")

```

```

        for paper in papers:
            title = paper.find('h3', class_='title').find('span')
            author = paper.find('a', class_='link person').find('span')
            date = paper.find('span', class_="date")
            link = paper.find('h3', class_='title').find('a', href=True)['href']

            opening = pd.read_csv('database.csv', index_col="Unnamed: 0")
            opening = opening.append({'Title': title.text,
                                     'Author': author.text,
                                     'Published': date.text,
                                     'Link': link}, ignore_index=True)
            opening.to_csv('database.csv')

```



```

def scrape(mx):
    df = update_csv('database.csv')
    i = 0
    while True:
        if i > mx:
            break

        if i > 0:
            url = URL + '?page=' + str(i)
        else:
            url = URL

        i = i + 1
        page = requests.get(url)
        soup = BeautifulSoup(page.content, "html.parser")
        results = soup.find(id="main-content")
        researchers = results.find_all("li", class_="grid-result-item")

        for researcher in researchers:
            check = researcher.find('div', class_='stacked-trend-widget')
            if check:
                name = check_department(researcher)
                if name is None:
                    pass
                else:
                    enter_each_researchers_publication(name, profile_url, df)

```

```
# create_csv()
mx = get_maximum_page()
scrape(mx)

# Indexing Component
# Indexing Component
scraped_db = pd.read_csv('database.csv', index_col=0)

def preprocess_text(text):
    # Convert text to lowercase
    text = text.lower()

    # Remove punctuation
    text = text.translate(str.maketrans("", "", string.punctuation))

    # Tokenization
    tokens = nltk.word_tokenize(text)

    # Remove stopwords
    stop_words = set(stopwords.words("english"))
    tokens = [token for token in tokens if token not in stop_words]

    # Lemmatization
    lemmatizer = WordNetLemmatizer()
    tokens = [lemmatizer.lemmatize(token) for token in tokens]
```

```
    return processed_text

def create_index():
    index = {}
    for i, row in scraped_db.iterrows():
        title = row['Title']
        author = row['Author']
        processed_title = preprocess_text(title)
        processed_author = preprocess_text(author)

        # Update index with title
        if processed_title not in index:
            index[processed_title] = []
        index[processed_title].append(i)

        # Update index with author
        if processed_author not in index:
            index[processed_author] = []
        index[processed_author].append(i)

    with open('index.json', 'w') as f:
        json.dump(index, f)
```

```

create_index()

import tkinter as tk
from tkinter import *
# Load the index from the JSON file

import tkinter as tk
from tkinter import *
import numpy as np
from sklearn.metrics import confusion_matrix

# Load the index from the JSON file
with open('index.json', 'r') as f:
    index = json.load(f)

def search_publications():
    query = entry.get(1.0, "end")
    processed_query = preprocess_text(query)

    if processed_query in index:
        matching_indices = index[processed_query]
        matching_publications = scraped_db.loc[matching_indices]

        # True positives: Matching publications retrieved and relevant
        true_positives = len(matching_publications)

```

```

# False negatives: Relevant publications not retrieved
false_negatives = len(scraped_db) - true_positives

# False positives: Non-relevant publications retrieved
false_positives = 0

# True negatives: Non-relevant publications not retrieved
true_negatives = 0

# Calculate false positives and true negatives
for i, row in scraped_db.iterrows():
    if i not in matching_indices:
        false_positives += 1
    else:
        true_negatives += 1

# Calculate the predicted and actual labels
predicted_labels = [1] * true_positives + [0] * false_positives
actual_labels = [1] * (true_positives + false_negatives) + [0] * (false_positives + true_negatives)

# Adjust the length of the lists if they are different
max_length = max(len(predicted_labels), len(actual_labels))
predicted_labels += [0] * (max_length - len(predicted_labels))
actual_labels += [0] * (max_length - len(actual_labels))

# Calculate the confusion matrix
cm = confusion_matrix(actual_labels, predicted_labels)

```

```

# Display the confusion matrix
messagebox.showinfo("Confusion Matrix", str(cm))

# Display the matching publications
messagebox.showinfo("Search Results", matching_publications.to_string())
else:
    messagebox.showinfo("Search Results", "No matching publications found.")

# Rest of the code...

# Create the main window
from tkinter import *
import tkinter as tk
window = tk.Tk()
window.title("Publication Search")
window.geometry("600x650")
window.configure(bg="#111D88")
window.resizable(0,0)
from PIL import ImageTk, Image
img=(Image.open("coventry-university-logo.png"))
imgg=img.resize((200,200))
imggg=ImageTk.PhotoImage(imgg)

lbl=Label(window,image=imggg,bg="#111D88")
lbl.pack(side=TOP)
# Create and position the search label
label = tk.Label(window, text="Enter author name or title name:",fg="white",font=("Arial", 20,"italic"),bg="#111D88")

```

```

label.pack(pady=10)

# Create and position the search entry
entry = tk.Text(window,height=10,width=30,font=("Arial", 20))
entry.pack()

# Create and position the search button
button = tk.Button(window, text="Search",font=("Arial", 20),bg="#10164B",fg="white",command=search_publications)
button.pack(pady=10)

# Start the main event loop
window.mainloop()

```

Text data.ipynb

```
import pandas as pd
```

```
training_dataset = pd.read_csv('news_df.csv')
```

```
print(training_dataset)
```

Unnamed: 0	Text	Class
0	UK economy facing 'major risks'\n\nThe UK manu...	business
1	Aids and climate top Davos agenda\n\nClimate c...	business
2	Asian quake hits European shares\n\n\nShares in ...	business
3	India power shares jump on debut\n\n\nShares in ...	business
4	Lacroix label bought by US firm\n\n\nLuxury good...	business


```
4          4  Lacroix label bought by US firm\n\nLuxury good...  business
...          ...
1292       1292 Bookmakers back Aviator for Oscar\n\nThe Aviat...  arts
1293       1293 Scissor Sisters triumph at Brits\n\nUS band Sc...  arts
1294       1294 Spears seeks aborted tour payment\n\nSinger Br...  arts
1295       1295 Angels 'favourite funeral song'\n\nAngels by R...  arts
1296       1296 DJ double act revamp chart show\n\nDJ duo JK a...  arts

[1297 rows x 3 columns]
```

```
for text,label in zip(training_dataset.Text, training_dataset.Class):
    print(f'CATEGORY: {label}')
    print('\n')
    print(f'TEXT: {text}')
    print('\n\n')
```

IR Coursework SearchEngine.ipynb

Package Installs

Some packages required for this task are by default not included in the jupyter notebooks package list. A pip-install of these packages is required.

```
# Delete the "#" before each subsequent line to install the package

#!pip install scrapy
#!pip install requests
#!pip install BeautifulSoup4
#!pip install nltk
#!pip install gensim
#!pip install xgboost
#!pip install pandastable
```

✓ 0.5s

Python

Imports

```
import requests
from bs4 import BeautifulSoup
import pandas as pd
import time
import datetime
import string
import json
import nltk
nltk.download('averaged_perceptron_tagger')
nltk.download('wordnet')
nltk.download('stopwords')
nltk.download('punkt')
from nltk import pos_tag
from nltk.corpus import wordnet
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
```

1. Crawler Component

```
# Define Seed Page
URL = "https://pureportal.coventry.ac.uk/en/organisations/coventry-university/persons/"

# Define profile URL format. This was obtained by manually examining the profile pages
profile_url = "https://pureportal.coventry.ac.uk/en/persons/"
```

```
def get_maximum_page():
    first = requests.get(URL)
    soup = BeautifulSoup(first.text, 'html.parser')
    final_page = soup.select('#main-content > div > section > nav > ul > li:nth-child(12) > a')[0]['href']
    fp = final_page.split('=')[-1]
    return int(fp)

mx = get_maximum_page()
```

```

def check_department(researcher):

    l1 = researcher.find('div', class_='rendering_person_short')

    for span in l1.find_all('span'):
        # Check department
        if span.text == str('School of Computing, Electronics and Maths'):
            name = researcher.find('h3', class_='title').find('span').text
            return name
        else:
            pass

# def create_csv():
#     database = pd.DataFrame(columns=['Title', 'Author', 'Published', 'Link'])
#     database.to_csv('database.csv')

def update_csv(database):
    current_data = pd.read_csv(database, index_col="Unnamed: 0")
    return current_data

def enter_each_researchers_publication(researcher, url, df):

    new_url = url + str(researcher).replace(' ', '-').lower() + '/publications/'
    page = requests.get(new_url)
    soup = BeautifulSoup(page.content, "html.parser")
    results = soup.find(id="main-content")
    papers = results.find_all("li", class_="list-result-item")

```

```

    for paper in papers:
        title = paper.find('h3', class_='title').find('span')
        author = paper.find('a', class_='link person').find('span')
        date = paper.find('span', class_="date")
        link = paper.find('h3', class_='title').find('a', href=True)['href']

        opening = pd.read_csv('database.csv', index_col="Unnamed: 0")
        opening = opening.append({'Title': title.text,
                                'Author': author.text,
                                'Published': date.text,
                                'Link': link}, ignore_index=True)
        opening.to_csv('database.csv')

```

```

## Scrape function
def scrape(mx):
    df = update_csv('database.csv')
    i=0
    while True:

        if i > 17:
            break

        if i>0:
            url = URL + '?page=' + str(i)
        else:
            url = URL

        i = i+1
        # scraping starts here
        page = requests.get(url)
        soup = BeautifulSoup(page.content, "html.parser")
        results = soup.find(id="main-content")
        researchers = results.find_all("li", class_="grid-result-item")

        for researcher in researchers:
            # Check if researcher has any papers
            check = researcher.find('div', class_='stacked-trend-widget')
            if check:
                name = check_department(researcher)
                if name is None:
                    pass
                else:
                    enter_each_researchers_publication(name, profile_url, df)

```

```
update_csv(database='database.csv') #create_csv
```

```
%time scrape(mx)
```

```

sample_db = pd.read_csv('database.csv').rename(columns={'Unnamed: 0':'SN'})
sample_db
print(f'{sample_db.shape[0]} records were scraped')

```

2. Indexing Component

```

scraped_db = pd.read_csv('database.csv').rename(columns={'Unnamed: 0':'SN'}).reset_index(drop=True)
scraped_db.head()
# scraped_db = pd.read_csv('database.csv', index_col=0)

```

```
sample_db.head(7)
```

```

single_row = scraped_db.loc[1,:].copy()
single_row

```

2.1 Preprocess Text

```
# Remove stop words
sw = stopwords.words("english")
lemmatizer = WordNetLemmatizer()

def tp1(txt):
    txt = txt.lower() # Make lowercase
    txt = txt.translate(str.maketrans('', '',
                                         string.punctuation)) # Remove punctuation marks
    txt = lemmatize(txt)
    return txt

def fwpt(word):
    tag = pos_tag([word])[0][1][0].upper()
    hash_tag = {"V": wordnet.VERB, "R": wordnet.ADV, "N": wordnet.NOUN, "J": wordnet.ADJ}
    return hash_tag.get(tag, wordnet.NOUN)
```

```
def lemmatize(text):
    tkns = nltk.word_tokenize(text)
    ax = ""
    for each in tkns:
        if each not in sw:
            ax += lemmatizer.lemmatize(each, fwpt(each)) + " "
    return ax
```

```
# Sample title
single_row['Title']
```

```
# Demonstration of lowercase and punctuation removal
tp1(single_row['Title'])
```

```
lemmatize(tp1(single_row['Title']))
#lemmatize(single_row['Title'])
```

```
sample_db['Title'].iloc[5]
```

```
scraped_db['Title'].iloc[5]
```


2.1.1 Preprocess entire dataframe

```
processed_db = scraped_db.copy()

def preprocess_df(df):
    df.Title = df.Title.apply(tp1)
    df.Author = df.Author.str.lower()
    df = df.drop(columns=['Author', 'Published'], axis=1)
    return df

preprocess_df(processed_db)
processed_db.head()
```

2.2 Index Construction

```
single = processed_db.loc[0,:].copy()
print(single)
indexing_trial = {}

words = single.Title.split()
SN = single.SN
word = words[0]
example = {word: [SN]}

print('=====')
print('Sample index')
print(example)
```

```
## Indexer Function
def apply_index(inputs, index):
    words = inputs.Title.split()
    SN = int(inputs.SN)
    for word in words:
        if word in index.keys():
            if SN not in index[word]:
                index[word].append(SN)
        else:
            index[word] = [SN]
    return index

indx = apply_index(inputs=single, index= {})
```



```

def full_index(df, index):
    for x in range(len(df)):
        inpt = df.loc[x,:]
        ind = apply_index(inputs=inpt, index=index)
    return ind

def construct_index(df, index):
    queue = preprocess_df(df)
    ind = full_index(df=queue, index=index)
    return ind

indexed = full_index(processed_db,
                      index = {})

indexes = construct_index(df=scraped_db,
                          index = {})

```

```

with open('indexes.json', 'w') as new_f:
    json.dump(indexes, new_f, sort_keys=True, indent=4)

with open('indexes.json', 'r') as file:
    data = json.load(file)

def index_2(df, x_path):
    if len(df) > 0:
        with open(x_path, 'r') as file:
            prior_index = json.load(file)
        new_index = construct_index(df = df, index = prior_index)
        with open(x_path, 'w') as new_f:
            json.dump(new_index, new_f, sort_keys=True, indent=4)

```

```
len(data)
```

✓ 0.0s

```
1258
```

```
data
```

✓ 0.3s

3. Query Processor

```
def demonstrate_query_processing():  
    sample = input('Enter Search Terms: ')  
    processed_query = tp1(sample)  
    #print(f'User Search Query: {sample}')  
    print(f'Processed Search Query: {processed_query}')  
    return processed_query
```

3.1. Split Query into individual terms

```
def split_query(terms):  
    each = tp1(terms)  
    return each.split()  
  
dqp = demonstrate_query_processing()  
dqp  
print(f'Split Query: {split_query(dqp)}')
```

3.2. Boolean Functionalities

```
def union(lists):  
    union = list(set.union(*map(set, lists)))  
    union.sort()  
    return union  
  
def intersection(lists):  
    intersect = list(set.intersection(*map(set, lists)))  
    intersect.sort()  
    return intersect
```

3.3. Search Engine Function

```
def vertical_search_engine(df, query, index=indexes):
    query_split = split_query(query)
    retrieved = []
    for word in query_split:
        if word in index.keys():
            retrieved.append(index[word])

    # Ranked Retrieval
    if len(retrieved)>0:
        high_rank_result = intersection(retrieved)
        low_rank_result = union(retrieved)
        c = [x for x in low_rank_result if x not in high_rank_result]
        high_rank_result.extend(c)
        result = high_rank_result

        final_output = df[df.SN.isin(result)].reset_index(drop=True)

        # Return result in order of Intersection ----> Union
        dummy = pd.Series(result, name = 'SN').to_frame()
        result = pd.merge(dummy, final_output, on='SN', how = 'left')
```

```
    else:
        result = 'No result found'

    return result
```

```
def test_search_engine():
    xtest = scraped_db.copy()
    query = input("Enter your search query: ")
    return vertical_search_engine(xtest, query, indexed)
```

```
test_search_engine()
```

```
def final_engine(results):
    if type(results) != 'list':
        return results
        #print(results)
    else:
        for i in range(len(results)):
            printout = results.loc[i, :]
```

```
scraped_db['Author'].iloc[24]
```

```
final_engine(test_search_engine())
```

4. Schedule Crawler for every week

To demonstrate a weekly scheduled crawling, the following parameters are defined:

- **interval** : Represents number of days in reality. In this code, it represents only seconds for demonstration

```
days = 0
interval = 7
while days <= 1:
    scrape(mx)
    print(f"Crawled at {datetime.datetime.now().strftime('%Y-%m-%d %H:%M:%S')}")
    print(f'Next crawl scheduled after {interval} days')
    time.sleep(interval)
    days = days + 1
```

5. GUI

```
from tkinter import *
from tkinter import messagebox
from PIL import Image, ImageTk
from tkinter import scrolledtext
from pandastable import Table, TableModel
from contextlib import suppress
import warnings
warnings.filterwarnings('ignore')

image1 = Image.open('coventry-university-logo.png')
resized_image1 = image1.resize((500,300), Image.ANTIALIAS)

def new_gui(image1):
    window = Tk()
    window.configure(bg='#0F1E9D')
    window.title("Coventry University")
    window.geometry('1100x600')

    lbl = Label(window, text="Search Engine",bg="#0F1E9D", font=("Arial Bold", 30), padx=5, pady=5)
    lbl.grid(column=1, row=0)
```

```
lbl2 = Label(window, text="Enter your search query here ==>", bg="#0F1E9D", font=("Arial", 15), padx=5, pady=5)
lbl2.grid(column=0, row=1)
```

```
img = ImageTk.PhotoImage(image1)
```

```
lbl3 = Label(image=img)
lbl3.image = img
lbl3.grid(column=1, row=3, padx=5, pady=5)
```

```
query = Entry(window, width=40)
query.grid(column=1, row=1, padx=5, pady=5)
```

```
results = Canvas(window, height=30, width=250)
results.grid(column=1, row=2, padx=5, pady=5)
```

```
# Entry
def getInputBoxValue():
    userInput = query.get()
    return userInput
```

```
def getInputBoxValue():
    userInput = query.get()
    return userInput
```

```
# Button
def clicked():
    search()
    #pass
```

```
def no_result():
    messagebox.showwarning("Warning", "No results found. Please try different search terms")
```

```
def search():
    xtest = scraped_db.copy()
    q = query.get()
    f = Frame(window)
    df = vertical_search_engine(xtest, q, indexed)
    if type(df) == str:
        no_result()
    else:
        pt = Table(results)
        try:
            table = pt = Table(results, dataframe=df)
            pt.show()
        except AttributeError:
            pass
```

```
def close_window():  
    if messagebox.askokcancel("Quit", "Quit Programme?"):  
        window.destroy()  
  
btn = Button(window, text="Search",bg="#0F1E9D", command=clicked)  
btn.grid(column=2, row=1)  
  
window.protocol("WM_DELETE_WINDOW", close_window)  
window.mainloop()
```

```
new_gui(resized_image1)
```


IR Coursework TextClassifier.ipynb

Imports

```
import os
import string

# Data Handling and Processing
import pandas as pd
import numpy as np
import re
from scipy import interp

# Visualuzation
import matplotlib.pyplot as plt
%matplotlib inline
%config InlineBackend.figure_format='retina'

# NLP Packages
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords, wordnet
from nltk.stem import WordNetLemmatizer
from nltk import pos_tag
import nltk
from joblib import dump, load
```

```
# ML Models
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import RidgeClassifier
from sklearn.tree import DecisionTreeClassifier

# Scikit Learn packages
from sklearn.base import clone
from sklearn.preprocessing import label_binarize, LabelEncoder
from sklearn.feature_extraction.text import TfidfVectorizer, CountVectorizer
from sklearn.model_selection import KFold, cross_validate, cross_val_score, train_test_split
from sklearn.pipeline import Pipeline
from sklearn import metrics
from sklearn.metrics import roc_curve, auc
```

Load Datasets

```
# business_path = 'bbc/business/'
# tech_path = 'bbc/tech/'
# arts_path = 'bbc/entertainment/'
# will be loaded from news_df.csv
```

✓ 0.0s

```
text_df = pd.DataFrame(columns=['Text', 'Class'])
# text_df.to_csv('news_df.csv')
text_df.head()
```

```
# Function to read the text files into one dataframe
def readfiles_to_dataframe(directory, category):
    arr = os.listdir(directory)
    strtext = ".txt"
    for textfile in arr:
        if textfile.__contains__(strtext):
            fileObject = open(directory + textfile, "r")
            data = fileObject.read()
            ouvert = pd.read_csv('news_df.csv', index_col="Unnamed: 0")
            ouvert = ouvert.append({"Class": str(category), "Text": data}, ignore_index=True)
            ouvert.to_csv('news_df.csv')

# Define categories
# paths = [business_path, tech_path, arts_path]
# categories = ['business', 'tech', 'arts']
```

```
full_df = pd.read_csv('news_df.csv')
print(full_df.shape)
full_df.head()
```

```
full_df.drop(columns=['Unnamed: 0'], inplace=True)
full_df.head()
```

1. Dataset Exploration

```
full_df['Class'].value_counts().plot(kind='bar')
plt.title('Number of News articles per Category', size=20, pad=20);
```

```
full_df.isna().sum()
```

2. Text Preprocessing

Here, unwanted parts of the text are removed such as special characters.

```
def preprocess(df):
    # Remove special characters
    df['Text2'] = df['Text'].replace('\n', ' ')
    df['Text2'] = df['Text2'].replace('\r', ' ')

    # Remove punctuation signs and lowercase all
    df['Text2'] = df['Text2'].str.lower()
    df['Text2'] = df['Text2'].str.translate(str.maketrans('', '', string.punctuation))

    # Remove stop words
    stop_words = stopwords.words("english")
    lemmatizer = WordNetLemmatizer()

    def fwpt(each):
        tag = pos_tag([each])[0][1][0].upper()
        hash_tag = {"N": wordnet.NOUN, "R": wordnet.ADV, "V": wordnet.VERB, "J": wordnet.ADJ}
        return hash_tag.get(tag, wordnet.NOUN)

    def lematize(text):
        tokens = nltk.word_tokenize(text)
        ax = ""
        for each in tokens:
            if each not in stop_words:
                ax += lemmatizer.lemmatize(each, fwpt(each)) + " "
        return ax

    df['Text2'] = df['Text2'].apply(lematize)
```

✓ 0.2s

```
preprocess(full_df)
```

Demonstration of Preprocessing

Original:

```
full_df.iloc[1]['Text']
```

```
full_df.iloc[1]['Text2']
```

3. Train Test Split

```
x_train, x_test, y_train, y_test = train_test_split(full_df['Text2'],  
                                                    full_df['Class'],  
                                                    test_size=0.2,  
                                                    random_state=9)
```

Check for acceptable category balance

```
y_train.value_counts().plot(kind='bar')  
plt.title('Category Balance', size=20, pad=20);
```

```
print(x_train.shape, x_test.shape, y_train.shape, y_test.shape)
```

4. Vectorize

```
vector = TfidfVectorizer(stop_words='english',  
                          ngram_range = (1,2),  
                          min_df = 3,  
                          max_df = 1.,  
                          max_features = 10000)
```

5. Construct Model

```
def fit_model(model, model_name):  
    line = Pipeline([('vectorize', vector), (model_name, model)])  
  
    output = cross_validate(line,  
                            X_train,  
                            y_train,  
                            cv = KFold(shuffle = True,  
                                       n_splits = 3,  
                                       random_state = 9),  
                            scoring = ('accuracy', 'f1_weighted', 'precision_weighted', 'recall_weighted'),  
                            return_train_score=True)  
    return output
```

```
dectree = fit_model(DecisionTreeClassifier(), 'DTree')  
ridge = fit_model(RidgeClassifier(), 'Ridge')  
bayes = fit_model(MultinomialNB(), 'NB')
```

```
dt = pd.DataFrame.from_dict(dectree)  
rc = pd.DataFrame.from_dict(ridge)  
bc = pd.DataFrame.from_dict(bayes)
```

```
l1 = [bc, rc, dt]  
l2 = ["NB", "Ridge", "DT"]  
  
for each, tag in zip(l1, l2):  
    each['model'] = [tag, tag, tag]  
  
joined_output = pd.concat([bc,rc,dt])
```

✓ 0.0s

dectree

ridge

✓ 0.0s

```
{'fit_time': array([0.95411253, 0.8628304 , 0.8784225 ]),  
 'score_time': array([0.37637424, 0.21873164, 0.22634482]),  
 'test_accuracy': array([0.97398844, 0.97976879, 0.9826087 ]),  
 'train_accuracy': array([1., 1., 1.]),  
 'test_f1_weighted': array([0.97402024, 0.9797116 , 0.982637 ]),  
 'train_f1_weighted': array([1., 1., 1.]),  
 'test_precision_weighted': array([0.97412924, 0.98002673, 0.98303811]),  
 'train_precision_weighted': array([1., 1., 1.]),  
 'test_recall_weighted': array([0.97398844, 0.97976879, 0.9826087 ]),  
 'train_recall_weighted': array([1., 1., 1.])}
```



```
bayes
```

```
relevant_measures = list(['test_accuracy', 'test_precision_weighted', 'test_recall_weighted', 'test_f1_weighted'])

dec_tree_metrics = joined_output.loc[joined_output.model == 'DT'][relevant_measures]
nb_metrics = joined_output.loc[joined_output.model == 'NB'][relevant_measures]
r_metrics = joined_output.loc[joined_output.model == 'Ridge'][relevant_measures]
```

Decision Tree metrics

```
dec_tree_metrics
```

Multinomial Naive Bayes metrics

```
nb_metrics
```

Ridge Classifier metrics

```
r_metrics
```

Average metrics

```
metrics_ = [dec_tree_metrics, nb_metrics, r_metrics]
names_ = ['Decision Tree', 'Naive Bayes', 'Ridge Classifier']

for scores, names in zip(metrics_, names_):
    print(f'{names} Mean Metrics:')
    print(scores.mean())
    print('  ')
```


Selection of Model

From the metrics obtained above, we see that **Ridge Classifier** performs best. However, the **Multinomial Naive Bayes classifier** is chosen to create the final model.

This is because it **has the ability to provide probability score** for each prediction it makes, while scoring similarly the best model.

```
# Join training and test datasets
x = pd.concat([x_train,
               x_test])
y = pd.concat([y_train,
               y_test])
```

```
def create_and_fit(clf, x, y):
    best_clf = clf
    pipeline = Pipeline([('vectorize', vector), ('model', best_clf)])
    return pipeline.fit(x, y)
```

```
# Create model
CLASSYfier = create_and_fit(MultinomialNB(), x, y)
```

`CLASSYfier.classes_`

```
## FINAL TESTING: ...
```

```
input_text = '(Reuters) -Netflix Inc said it would make a deeper dive into video games as the movie and TV streaming service'
CLASSYfier.predict_proba([input_text])
```

```
CLASSYfier.predict([input_text])[0]
```

GUI

```
import tkinter as tk
from tkinter.scrolledtext import ScrolledText

window = tk.Tk()

window.title("TEXT CLASSIFIER")
window.minsize(600,400)

text_box = ScrolledText(window)
text_box.grid(column=0, row=1, padx=5, pady=5)

def result(res, pr):
    ARTS = round(pr[0][0], 3)
    BUSINESS = round(pr[0][1], 3)
    TECH = round(pr[0][2], 3)

    lines = [f"Arts: {ARTS}", f"Business: {BUSINESS}", f"Tech: {TECH}"]
    tk.messagebox.showinfo(message= f"Predicted Category: {str(res).capitalize()}" + "\n\n" + "\n".join(lines))

def clickMe():
    classification = tk.StringVar()
    category, probabilities = classify_text(text_box.get("1.0",tk.END))

def classify_text(input_text):
    out = CLASSYfier.predict([input_text])[0]
    probs = CLASSYfier.predict_proba([input_text])
    return out,probs

label = tk.Label(window, text = "Enter Text to be classified")
label.grid(column = 0, row = 0)

btn = tk.Button(window, text="Classify", command=clickMe)
btn.grid(column=0, row=2)

window.mainloop()
```